

API Rate Limiter + Backend System

Token Bucket Based Rate Limiting using Spring Boot

RATE LIMITING PROTECTING BACKEND



What is Rate Limiting?

- Each user can send limited requests
- If limit is crossed → request is blocked

Example:

- Limit = 5 requests
- 6th request → rejected

Token Bucket Concept

How it works:

- Each user has a bucket
- Bucket contains tokens
- Each request uses 1 token
- Tokens refill over time

Example:

Capacity = 5

Refill = 5 token/min

Request	Result
1–5	Allowed
6	Rejected(429)

Token Bucket Algorithm

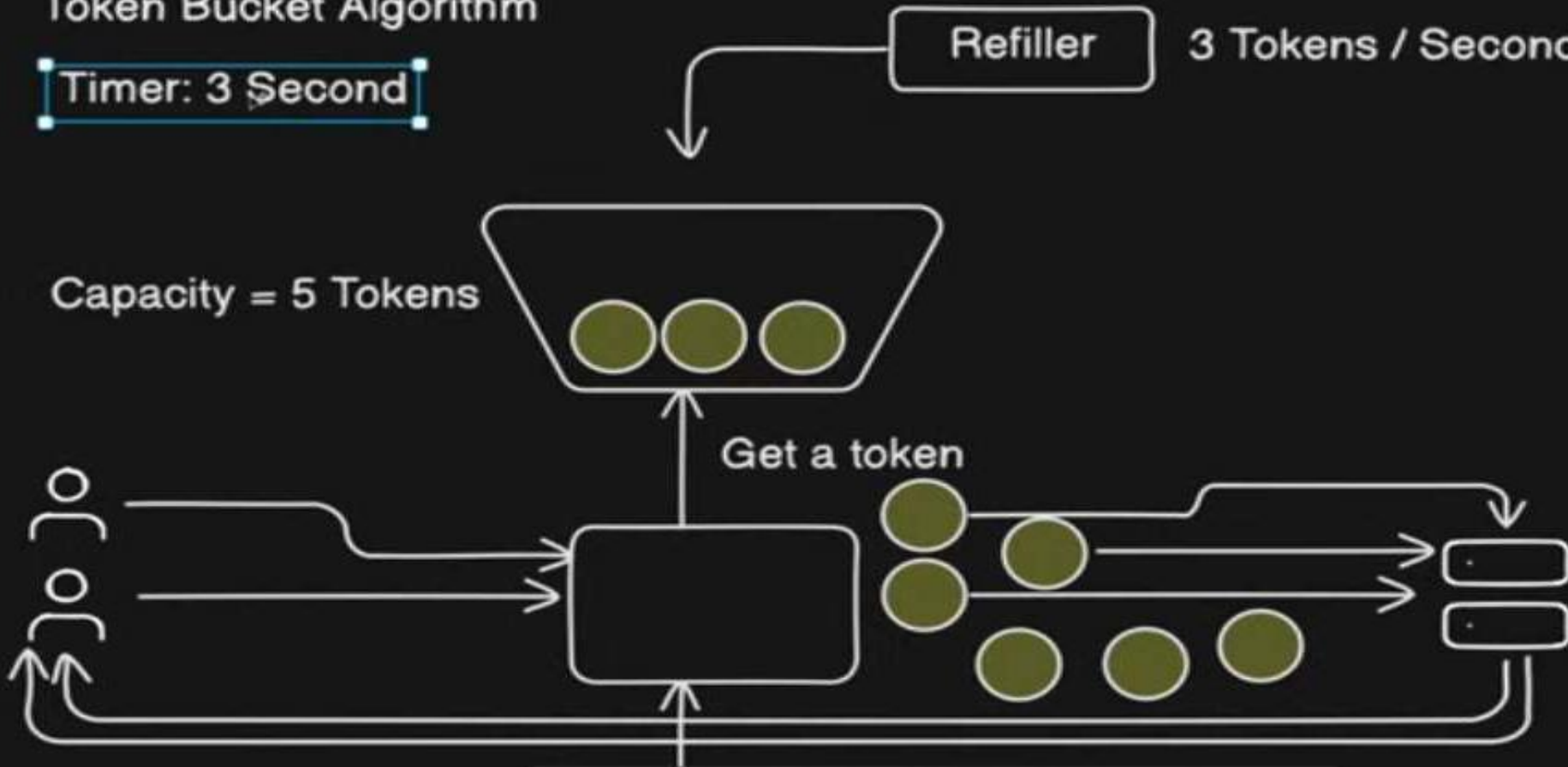
Timer: 3 Second

Refiller

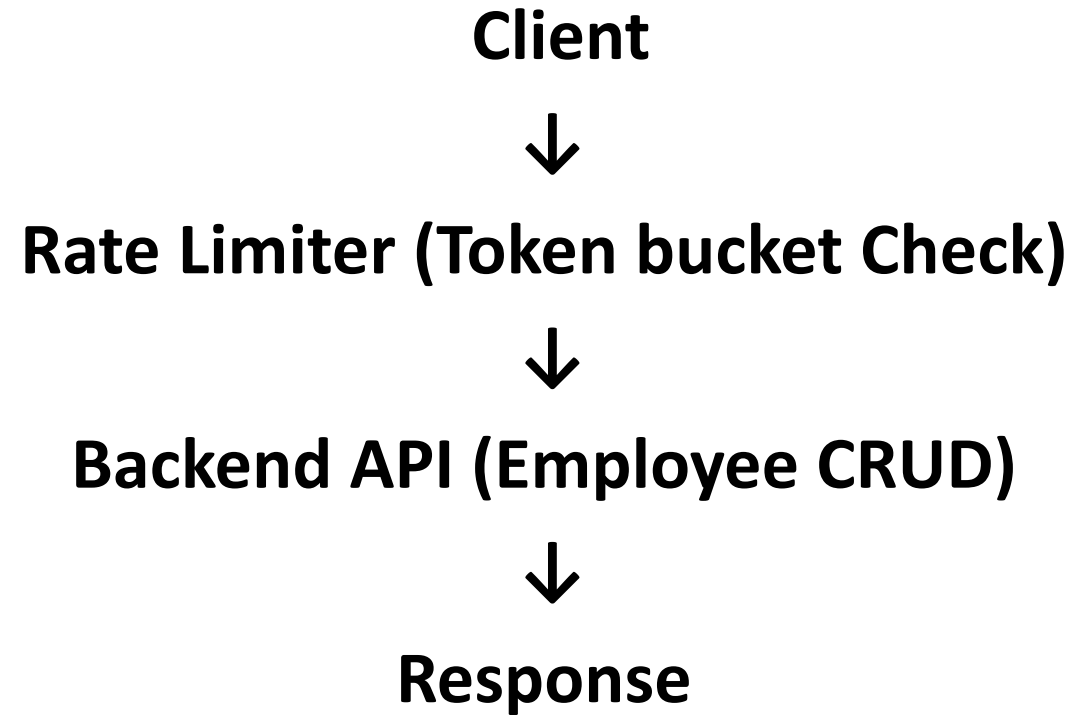
3 Tokens / Second

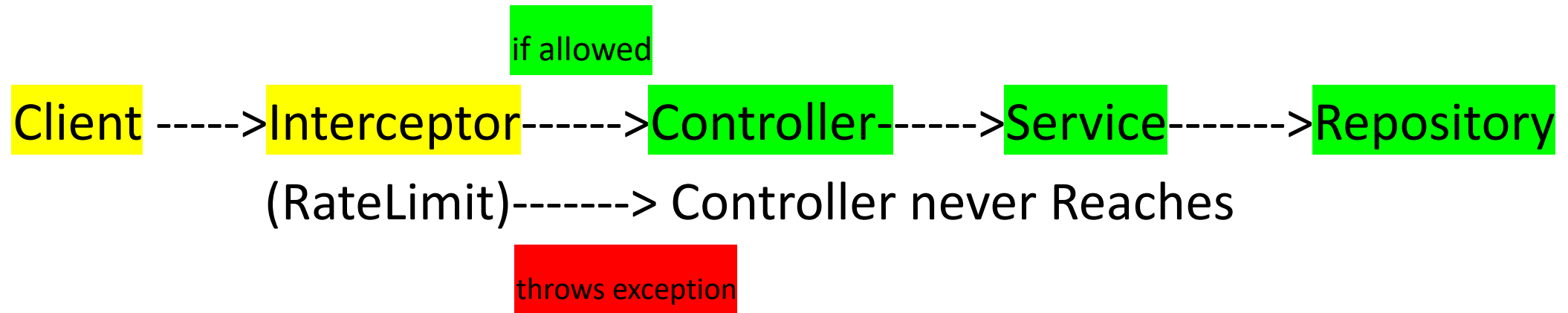
Capacity = 5 Tokens

Get a token

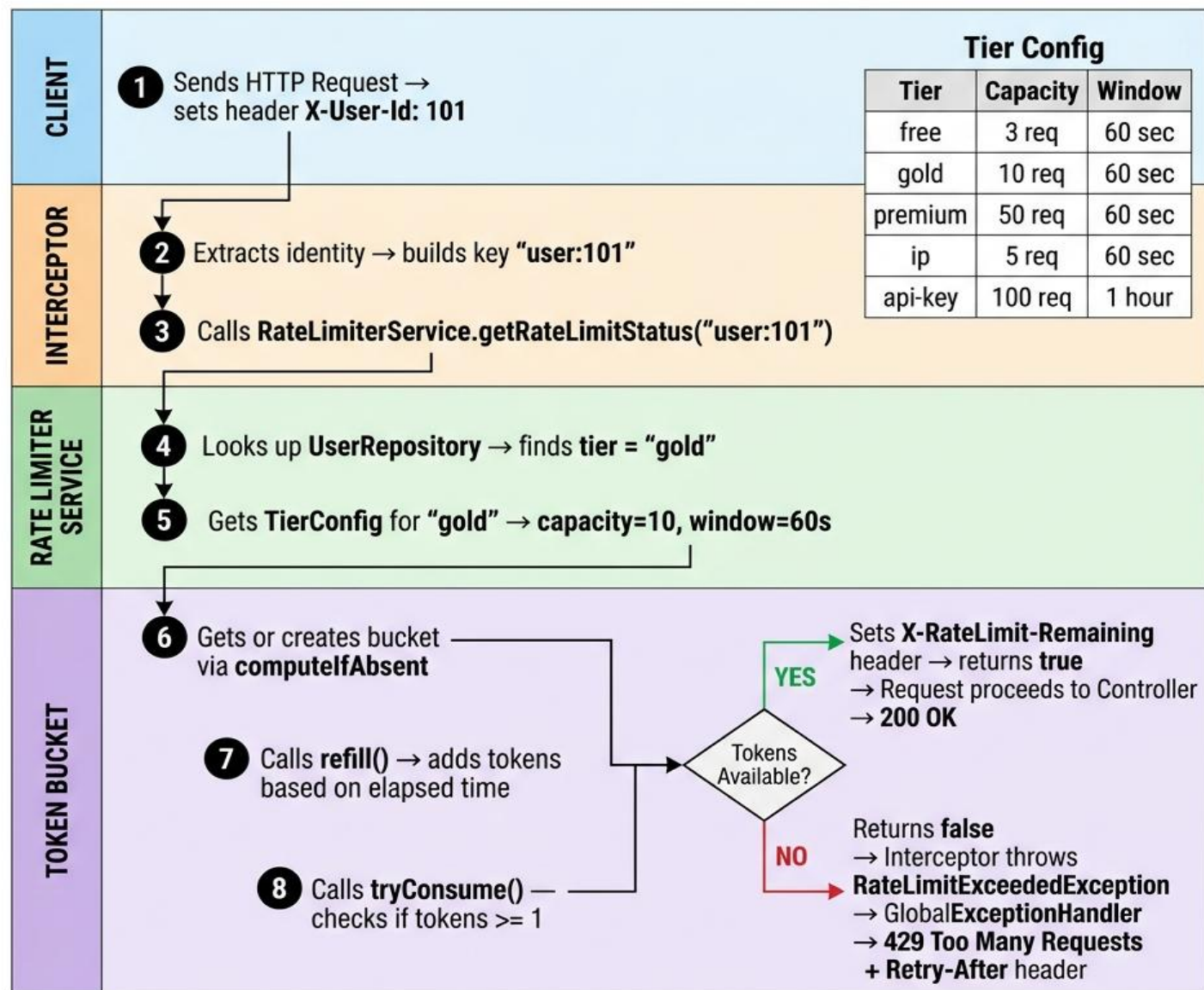


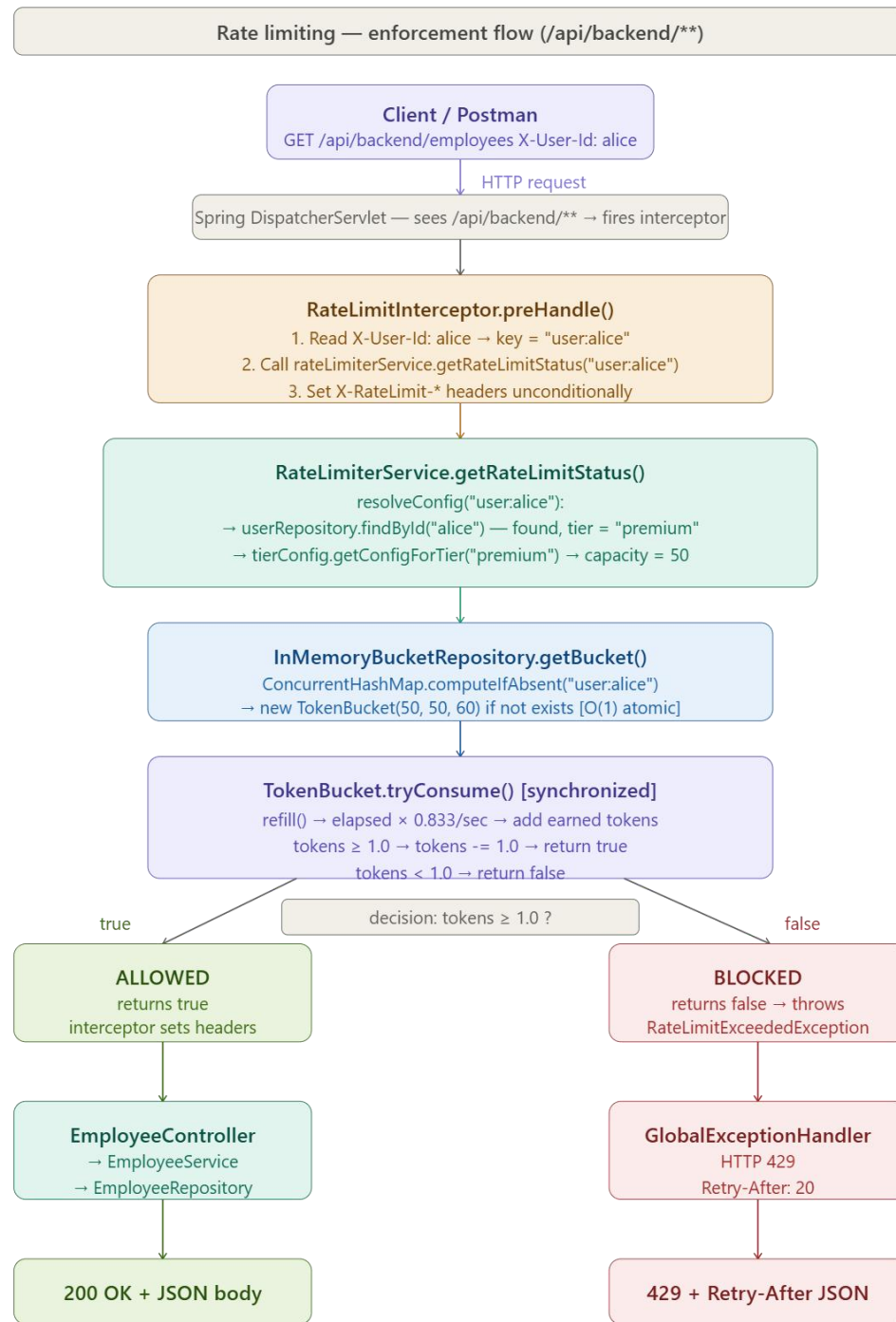
Overview:





Rate Limiting Flow — Token Bucket Algorithm





Employee management API — all endpoints under /api/backend/employees/**

RateLimitInterceptor fires first (always)

WebConfig registered on /api/backend/** — runs before any controller

Token check passes → interceptor returns true → controller runs

EmployeeController (@RequestMapping /api/backend/employees)

Zero rate-limiting code here — @Valid triggers Bean Validation automatically

GET all

/employees
returns list

GET by ID

/employees/1
200 or 404

POST create

@Valid fires
201 or 400

PUT / DELETE

update / remove
200 or 404

EmployeeService

Business logic — throws ResourceNotFoundException if employee not found

EmployeeRepository (ConcurrentHashMap)

Seeded at startup: Alice Johnson · Bob Martinez · Carol Williams

GlobalExceptionHandler (@RestControllerAdvice)

ResourceNotFoundException → 404 MethodArgumentNotValidException → 400
RateLimitExceededException → 429 Exception → 500

200 / 201 OK

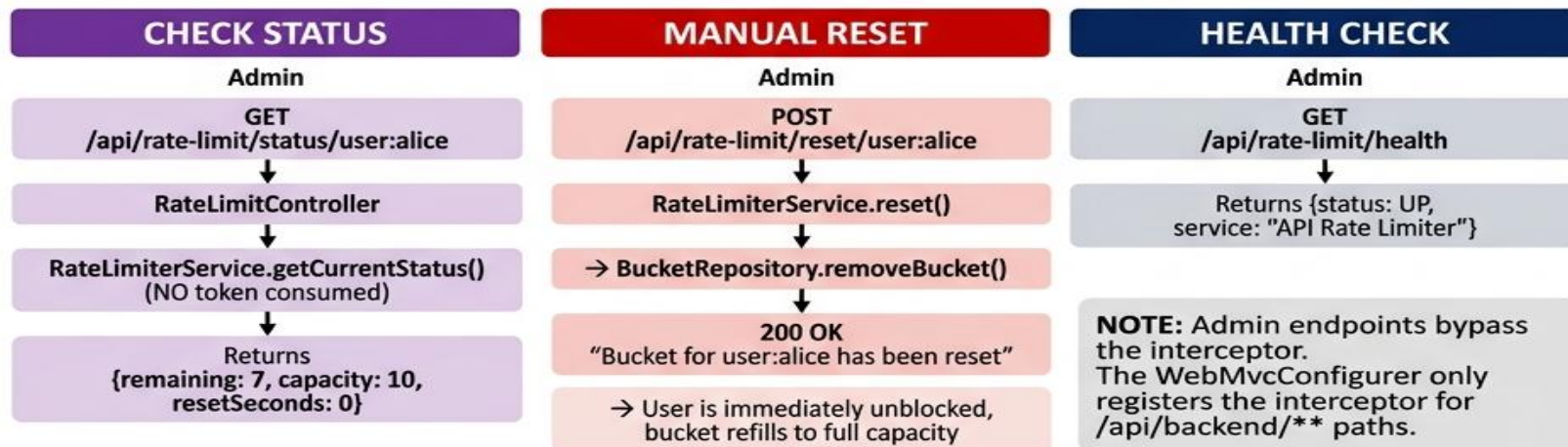
+ X-RateLimit-* headers on every response

400 / 404 / 429 error

structured JSON body + Retry-After on 429

NOT rate-limited — /api/users/** (Admin Controlled)

/api/rate-limit/ (No Rate Limiting Applied)**



Token Bucket Dry Run — Free Tier

capacity = 3 · refillTokens = 3 · windowSeconds = 60 · refillRatePerSecond = 0.05/sec

REQ	TIME	ELAPSED	TOKENS BEFORE REFILL	TOKENS AFTER REFILL	AFTER CONSUME	RESULT
1	t = 0s	0 – 0 = 0s	3.0 starts full	3.0 0 × 0.05 = 0 added	2.0 3.0 – 1.0	✓ 200
2	t = 1s	1 – 0 = 1s	2.0 left from req 1	2.05 1 × 0.05 = 0.05 added	1.05 2.05 – 1.0	✓ 200
3	t = 2s	2 – 1 = 1s	1.05 left from req 2	1.10 1 × 0.05 = 0.05 added	0.10 1.10 – 1.0	✓ 200
4	t = 3s	3 – 2 = 1s	0.10 left from req 3	0.15 1 × 0.05 = 0.05 added	— blocked 0.15 < 1.0	✗ 429 Retry-After 17s
Retry-After = Math.ceil((1.0 – 0.15) / 0.05) = Math.ceil(17.0) = 17 seconds						
5	t = 20s	20 – 3 = 17s	0.15 still from req 4	1.0 17 × 0.05 = 0.85 added	0.0 1.0 – 1.0	✓ 200
6	t = 80s	80 – 20 = 60s	0.0 from req 5	3.0 60 × 0.05 = 3.0 → capped at 3	2.0 3.0 – 1.0	✓ 200 burst ready

tokensToAdd = elapsedSeconds × refillRatePerSecond
tokens = Math.min(capacity, tokens + tokensToAdd)
Retry-After = Math.ceil((1.0 – tokens) / refillRatePerSecond)

- **RateLimitServiceTest**

- `shouldBlockAfterCapacityReached()`: Ensures bucket respects capacity limit strictly
- `shouldReturnCorrectStatus()` :Service correctly wraps bucket logic + exposes API-friendly response
- `shouldResetBucket()` :Reset restores full capacity (important for admin/manual override cases)
- `shouldThrowWhenKeyIsNull()`: I/p validation
- `shouldThrowWhenKeyIsBlank()`: I/p validation

- **BackendControllerTest.java**

- `shouldAllowBackendAccessAndIncludeHeaders():` Interceptor allows request AND injects rate-limit headers
- `shouldBlockWhenLimitExceeded():` Interceptor short-circuits request — controller is NEVER called
- `shouldReturn404ForNonExistentUser():` Rate limiter allows request, but business logic fails → 404

- **RateLimiterControllerTest.java**

- `shouldGetStatus:` NOT testing logic, only HTTP mapping + JSON response
- `shouldResetBucket()`

Key Components

Main Parts:

- TokenBucket → logic
- RateLimiterService → decision maker
- InMemoryBucketRepository → stores buckets
- RateLimiterInterceptor → gatekeeper
- GlobalExceptionHandler → Handle Errors

Backend System (Employee API)

APIs:

- Create Employee
- Get Employee
- Update Employee
- Delete Employee

Layers:

Controller



Service



Repository

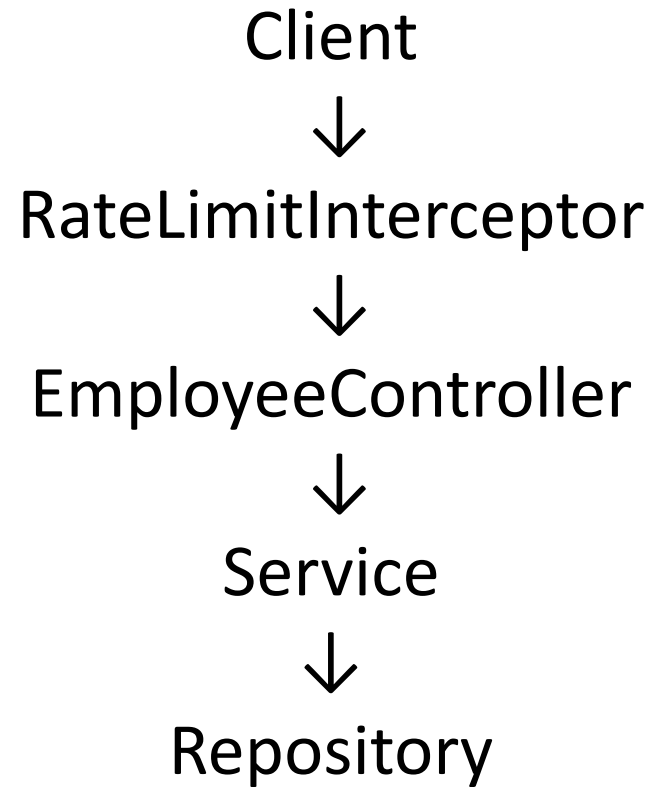


Model

Integration:(Expected Architecture)

How Rate Limiter Protects Backend?

WebConfig → registers interceptor → applies to /api/backend/**



Limitations

- Uses in-memory storage
- Data lost on restart
- Not suitable for multiple servers

Error Handling

If limit exceeded:

- Return 429 Too Many Requests

Headers:

- Retry-After
- Remaining tokens

Final Summary

What I built?

- Rate limiter using Token Bucket
- Thread-safe system
- Middleware-based design
- Integrated with backend

Why Token Bucket?

Advantages:

- Allows small bursts
- Smooth traffic control
- Simple and fast

Real World Examples

Where this is used:

- Payment APIs (to avoid misuse)
- Login systems (to prevent brute force)
- Public APIs (to ensure fair usage)

Problem Statement & Goals

PROBLEMS:

- APIs can be overloaded if too many requests come
- One user can send unlimited requests
- This can slow down or crash the system

GOALS:

- Control number of requests
- Ensure fair usage
- Protect backend system

$$\text{refillRatePerSecond} = (\text{double}) \text{refillTokens} \div \text{windowSeconds}$$

Tier	capacity	refillTokens	windowSeconds	= rate/sec	1 token every	Retry-After (empty)
FREE	3	3	60	0.0500	20 sec	~17 sec
GOLD	10	5	60	0.0833	12 sec	~12 sec
PLATINUM	50	50	60	0.8333	1.2 sec	~2 sec
IP	5	5	60	0.0833	12 sec	~12 sec
API key	100	100	3600	0.0278	36 sec	~36 sec

All values come from application.yaml — change any number, restart, zero code change.

FREE tier

capacity3 tokens

refillTokens3

windowSeconds60 s

refillRatePerSecond3 ÷ 600.0500 tokens/sec

1 full token every1 ÷ 0.05 = 20 seconds

Dry run — window 60 s (3 requests in ~3 seconds)

Request	Time	Tokens before refill	Elapsed	Tokens added	After consume	Result
1	t=0s	3.00	0s	0.00	2.00	200 ✓
2	t=1s	2.00	1s	+0.05	1.05	200 ✓
3	t=2s	1.05	1s	+0.05	0.10	200 ✓
4	t=3s	0.10	1s	+0.05	0.15 < 1.0	429 ✗ Retry-After: 17s
5	t=20s	0.15	17s	+0.85	0.00	200 ✓ (auto-refilled)

Same FREE tier — window changed to 3600 s (per-hour limit)

windowSeconds 3600 s (1 hour)

refillRatePerSecond $3 \div 3600 = 0.000833$ tokens/sec

1 full token every 1200 seconds = 20 minutes

Retry-After after 4 requests ~1180 seconds $\approx$ 20 minutes

Request	Time	Tokens before refill	Elapsed	Tokens added	After consume	Result
1	t=0s	3.00	0s	0.000	2.00	200 ✓
2	t=1s	2.00	1s	+0.0008	1.001	200 ✓
3	t=2s	1.001	1s	+0.0008	0.002	200 ✓
4	t=3s	0.002	1s	+0.0008	0.003 < 1.0	429 ✗ Retry-After: 1197s
5	t=1200s	0.003	1197s	+0.997	0.00	200 ✓ (waited 20 min)

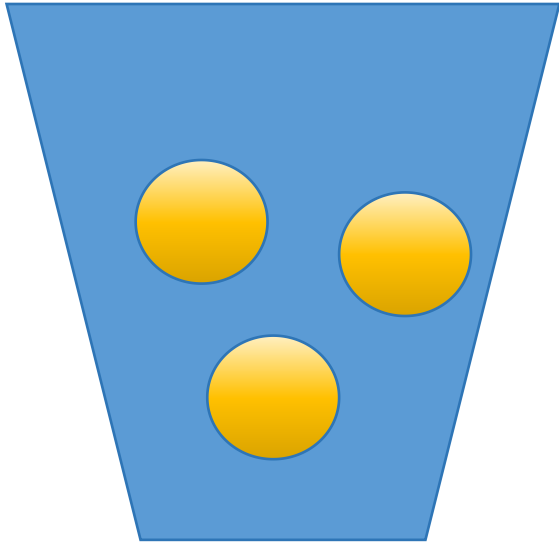
Changing only windowSeconds from 60 → 3600 makes each token take 20 minutes to earn back. Zero code change.

Configuration	Capacity	Refill Tokens	Window	Tokens/Sec	1 Token Every
Free Tier A	3	3	60 sec	0.05	20 sec
Free Tier B	3	3	3600 sec	0.000833	20 min

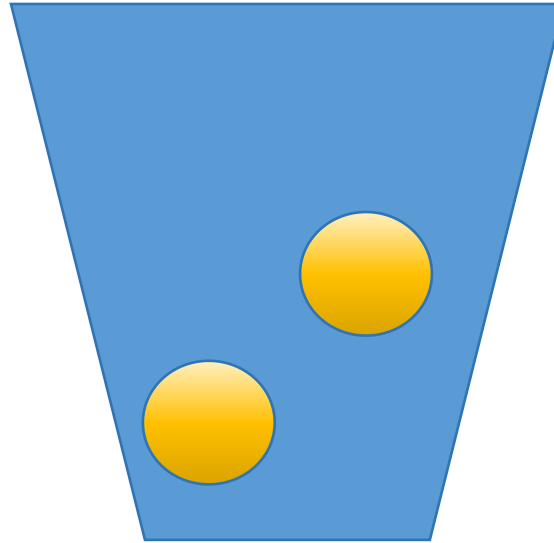
Window	Refill Speed
10 sec	1 token every 3.3 sec
60 sec	1 token every 20 sec
3600 sec	1 token every 20 min

- FREE TIER: (1 request consumes one token)

Initially (Full capacity)



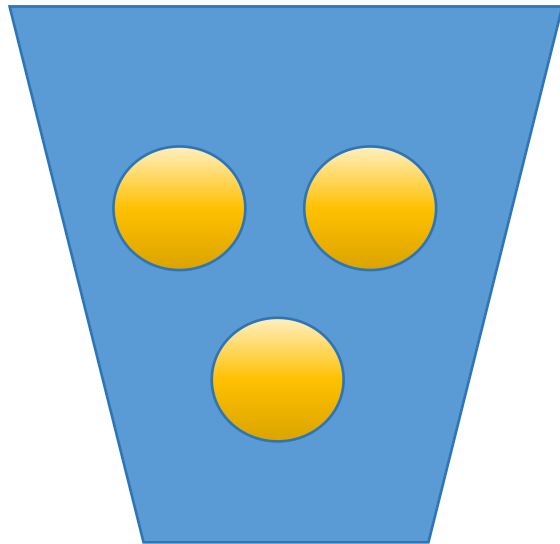
AFTER 1 request



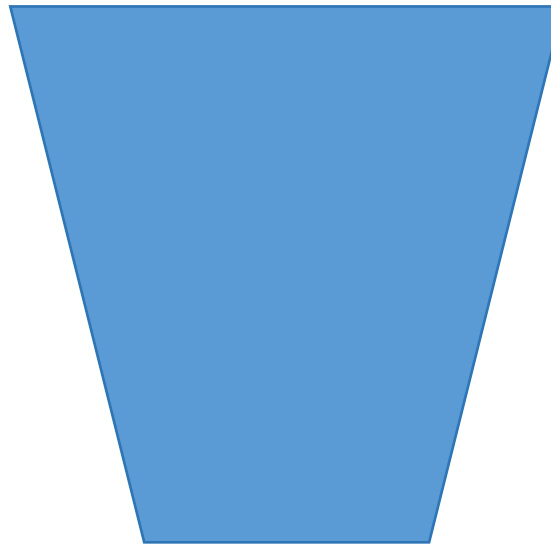
$\text{RefillPerSecond} = \text{double}(\text{refillToken}) / \text{WindowSec}$

Bucket for Free Tier

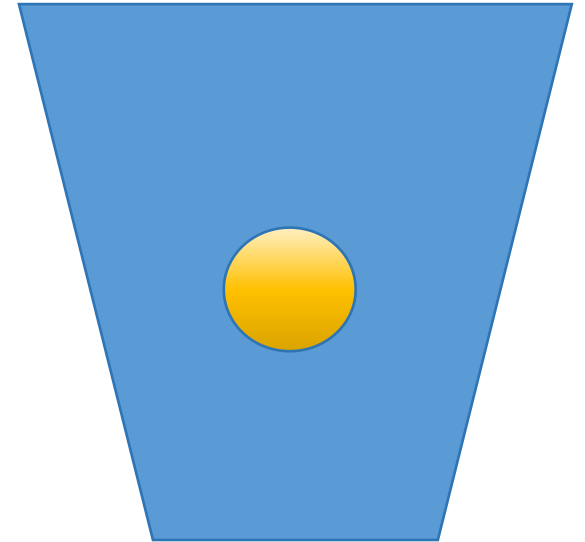
RefillToken:3 ,WindowSec:60



After 3 requests in 3 sec



After waiting for
17 sec



For FREE TIER:

RefillToken:3 ,WindowSec:60

$\text{RefillPerSecond} = \text{double}(\text{refillToken}) / \text{WindowSec}$

$\text{RefillPerSecond} = 3/60 \sim 0.05$ tokens per second

#Send 3 request in 3 seconds ($3 \times 0.05 = 0.15$ tokens added)

$\text{tokensToAdd} = \text{elapsedSeconds} \times \text{refillRatePerSecond}$

For one complete token

Need Extra 0.85 tokens ($0.85/0.05 = 17$)

RETRY-AFTER time will be 17 seconds

It shows that 1 tokens is gained in this time period.